Technology Stack Rationale

> *Prepared for: Investor Tech Audit Workshop Date: 2026-03-29 | Version: 1.0*

---

1. Technology Selection Framework

Each technology was evaluated against four criteria:

1. **Development velocity** -- How fast can we ship features?
2. **Operational simplicity** -- How much DevOps overhead does it create?
3. **Cost efficiency** -- What are the marginal costs at scale?
4. **Extensibility** -- How easy is it to adapt as requirements change?

---

2. Core Framework: Next.js 14 (App Router)

Why Next.js

Factor	Assessment
SSR + API in one	Eliminates need for separate backend. 132 API routes and 60+ pages in a single deployable unit.
App Router	React Server Components reduce client-side JavaScript. Server-rendered pages for SEO (blog, library). Client components for interactive features (recording, games).
Vercel native	Zero-config deployment. Push to git = deployed in 60 seconds.
TypeScript first	Full-stack type safety from database (Prisma) to UI.
Ecosystem	NextAuth, next-themes, next-intl -- mature middleware ecosystem.

Alternatives Considered

Alternative	Why Not
Remix	Smaller ecosystem, less Vercel integration at time of selection
SvelteKit	Smaller talent pool, fewer UI component libraries
Express + React SPA	Two deployments, SSR requires additional setup
Django + React	Python/JS boundary adds complexity, separate deployment

Trade-offs Accepted

- **10-second function timeout** on Vercel Hobby (mitigated with `maxDuration` config on AI-heavy routes)
- **Cold starts** on serverless (~200-500ms, acceptable for learning app)
- **No WebSockets** (real-time features use polling; WebSocket requires separate infrastructure)

---

3. Mobile: React Native (Expo SDK 52)

Why Expo + React Native

Factor	Assessment
Shared TypeScript	Same language as web app. Types, utilities, and API contracts shared.
Native camera/audio	Required for recording exercises. Expo Camera and AV modules.
Expo Router	File-based routing mirrors Next.js App Router conventions.
OTA updates	Push updates without App Store review cycle.
Expo Go	SDK 52 supports Expo Go for rapid development iteration.

Alternatives Considered

Alternative	Why Not
Flutter	Dart requires separate skill set, no TypeScript sharing
PWA	Camera/recording APIs unreliable on iOS Safari
Swift + Kotlin	Two separate codebases, 2x development time
Capacitor	Less mature than Expo for camera-heavy apps

Trade-offs Accepted

- **Expo SDK 52** (not 53) chosen for Expo Go compatibility -- SDK 53 requires dev client builds
- **No offline-first** -- requires network for API calls (recording uploads, AI feedback)
- **Larger binary** than pure native (~30MB vs ~5MB)

---

4. Database: Prisma 6 + Neon PostgreSQL

Why Prisma + Neon

Factor	Assessment
Type-safe ORM	Prisma generates TypeScript types from schema. Autocomplete on every query.
45 models, zero SQL	Complex schema maintained in readable `.prisma` DSL. Relationships, indexes, constraints -- all declarative.
Neon serverless	PostgreSQL without a persistent server. Connection pooling (PgBouncer) built in.
Scale-to-zero	Compute pauses when inactive. Cost-efficient for early stage.
Branching	Database branches for development -- like git branches for your database.

Alternatives Considered

Alternative	Why Not
Supabase	Good, but Prisma ORM preferred over raw SQL for type safety

PlanetScale (MySQL)	MySQL less suitable for JSON operations; PlanetScale dropped free tier
MongoDB	Relational data (users, subscriptions, progress) better suited for SQL
Firebase Firestore	NoSQL, complex queries difficult, vendor lock-in
Drizzle ORM	Newer, less battle-tested at time of selection; good future option

Trade-offs Accepted

- `prisma db push` instead of `prisma migrate` -- faster iteration, but no migration history (known tech debt)
- `(prisma as any)` **cast pattern** -- workaround when `prisma generate` cannot run locally
- **Cold start penalty** -- Prisma client initialization adds ~100ms on cold starts

---

5. AI Services: OpenAI (GPT-4o, Whisper, TTS)

Why OpenAI

Factor	Assessment
GPT-4o for coaching	Best-in-class instruction following for personalized feedback.
Whisper for STT	Accurate speech-to-text for voice chat with Coach Ney.
TTS for voice	Natural voice output for AI assistant responses.
GPT-4o-mini	Cost-efficient for high-volume tasks (motivation messages, crawl scoring).
JSON mode	`response_format: { type: 'json_object' }` for structured output.

Usage by Feature

Feature	Model	Cost Sensitivity
Coach Ney chat	GPT-4o	Medium (plan-gated)
Full performance feedback	GPT-4o (optional)	Low (MediaPipe primary)
Crawl relevance scoring	GPT-4o-mini	Low (admin-initiated)
Motivation messages	GPT-4o-mini	Medium (daily per user)
Observation guides	GPT-4o	Low (one-time per clip)
Speech-to-text	Whisper	Medium (voice chat)
Text-to-speech	TTS	Medium (voice responses)

Alternatives Considered

Alternative	Why Not
Claude (Anthropic)	Strong contender; OpenAI chosen for Vision capabilities initially

Gemini (Google)	Good multimodal, but less mature API ecosystem at selection time
Local models	Require GPU infrastructure; not feasible on serverless
Replicate (hosted)	Higher latency, less consistent quality for coaching text

---

6. Client-Side ML: Google MediaPipe

Why MediaPipe

Factor	Assessment
Zero API cost	ML runs entirely in the browser. No server calls for scoring.
Real-time	Face + pose detection at 30fps in browser.
Two models	FaceLandmarker (468 landmarks) + PoseLandmarker (33 landmarks).
CDN-loaded	Models loaded from Google CDN, no bundle size impact.

Cost Impact

This is arguably the most important technical decision in the stack:

```
Without MediaPipe (GPT-4o Vision for every recording):
  30K DAU x 5 recordings/day x $0.01/call = $1,500/day = $45,000/month

With MediaPipe:
  $0/month for all Arcade + Micro-Practice scoring
  GPT-4o only for optional Full Performance text feedback
```

Annual savings at 100K users: ~$540,000

Alternatives Considered

Alternative	Why Not
TensorFlow.js	Lower-level, requires custom model training
GPT-4o Vision	$0.01/call, unsustainable at scale (was original implementation)
Custom ML model	Requires training data, infrastructure, expertise
Face-api.js	Older, less accurate, no pose detection

---

7. Deployment: Vercel

Why Vercel

Factor	Assessment
Zero-config	`git push` = deployed. No Dockerfile, no CI/CD pipeline to maintain.

Global CDN	Edge network for static assets and SSR pages.
Serverless functions	Each API route auto-scales independently.
Blob storage	Integrated file storage for recordings and images.
Preview deployments	Every PR gets a unique preview URL.
Cron jobs	Built-in cron scheduling (2 jobs on Hobby plan).

Alternatives Considered

Alternative	Why Not
AWS (ECS/Lambda)	Higher complexity, requires DevOps expertise
Railway	Good, but less mature than Vercel for Next.js
Fly.io	Container-based, more operational overhead
Cloudflare Pages	Limited Next.js compatibility at time of selection
Self-hosted	Requires infrastructure team, defeats rapid iteration goal

Trade-offs Accepted

- **Hobby plan limits**: 10s default timeout (extended with `maxDuration`), 2 cron jobs max
- **Vendor lock-in**: Deep integration with Vercel Blob, Cron, Edge -- migration requires effort
- **No persistent processes**: WebSocket, background workers require separate services

---

8. Styling: Tailwind CSS 3.4

Why Tailwind

Factor	Assessment
Rapid iteration	Utility classes enable UI changes without CSS file management.
Consistent design	Built-in spacing, color, typography scales.
Dark mode	`dark:` variant for theme switching (transitioning to light mode).
Bundle size	Tree-shaken CSS -- only used utilities in final bundle.
Component patterns	`clsx` + `tailwind-merge` for conditional class composition.

Alternatives Considered

Alternative	Why Not
CSS Modules	More boilerplate, slower iteration
styled-components	Runtime CSS-in-JS, performance overhead
Chakra UI	Opinionated component library, less control

shadcn/ui	Good complement (could adopt for component library)

---

9. Supporting Libraries

Library	Version	Purpose	Rationale
`next-auth`	4.24	Authentication	De facto standard for Next.js auth
`bcryptjs`	3.0	Password hashing	Industry standard, pure JS (no native deps)
`jsonwebtoken`	9.0	Mobile JWT	Lightweight, well-tested
`zod`	3.23	Input validation	TypeScript-native schema validation
`openai`	4.47	OpenAI SDK	Official SDK, streaming support
`resend`	6.9	Email delivery	Modern email API, great DX
`twilio`	5.13	WhatsApp messaging	Industry standard messaging API
`web-push`	3.6	Push notifications	W3C Push API implementation
`@vercel/blob`	0.23	File storage	Native Vercel integration
`@mediapipe/tasks-vision`	0.10	ML inference	Google's official client-side ML
`youtube-transcript`	1.3	Transcript fetching	Simple API for YouTube captions
`lucide-react`	0.378	Icons	Lightweight, tree-shakeable icon set
`jszip`	3.10	ZIP import	Admin clip import from archives
`clsx`	2.1	Class composition	Conditional className merging
`tailwind-merge`	2.3	Class deduplication	Resolves Tailwind class conflicts

---

10. What We Would Change (Honest Assessment)

Current Choice	Would Reconsider	Why
Prisma `db push`	Prisma Migrate	Need migration history for production safety
No test framework	Jest + Playwright	44K LOC without tests is a risk
NextAuth v4	Auth.js v5 (NextAuth v5)	v5 has better App Router integration
Custom gamification	Existing library	7 gamification modules is significant custom code
DB notification queue	Redis + BullMQ	Better throughput and reliability at scale
2 cron jobs	Dedicated scheduler	Vercel Hobby limits to 2; need more timezone coverage
No caching layer	Redis/Upstash	Every request hits Neon; caching would reduce latency

---

11. Stack Maturity Matrix

```
Legend: [===] Mature  [== ] Growing  [=  ] Early  [   ] Planned

Core Web App        [===] 44K LOC, 60+ pages, production
API Layer           [===] 132 endpoints, consistent patterns
Database Schema     [===] 45 models, 62 indexes, 915 lines
Gamification        [===] 7 modules (XP, streaks, hearts, quests, badges, leaderboard, combo)
Learning Engine     [== ] 4 channels, 3 analyzers, 3 planners -- new (M47-M50)
AI Content Gen      [== ] 15 providers, image + video -- production but lightly tested
Client ML           [===] MediaPipe face + pose, production since M25
Mobile App          [=  ] 28 screens scaffolded, connected to API, not in stores
Payments            [=  ] PayPal + VNPay sandbox integrated, not live
Testing             [   ] No automated test suite (known debt)
Monitoring          [=  ] Error logging exists, no APM or alerting
```

Document prepared by Data Engineer role. Technology assessments based on codebase review and development history as of 2026-03-29.